INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA
SALT LAKE
(Affiliated to UEM Kolkata and Approved by AICTE)

Advanced Programming (OOP) Laboratory Project Report

[PCCCS495]

Course Instructors: Prof. Dr. Swagatam Basu, Prof. Dr. Susovan Jana, Prof. Dr. Soumadip Biswas

Project Title: Smart Course Registration & Academic Planner System

YEAR: 2ND SEMESTER: 4

NAME: **SREETAMA SANTRA**

ROLL NO: **65** STREAM: **CSE (IoT)**

ENROLLMENT NO: **12024052019052**

Spring 2026

PROBLEM STATEMENT

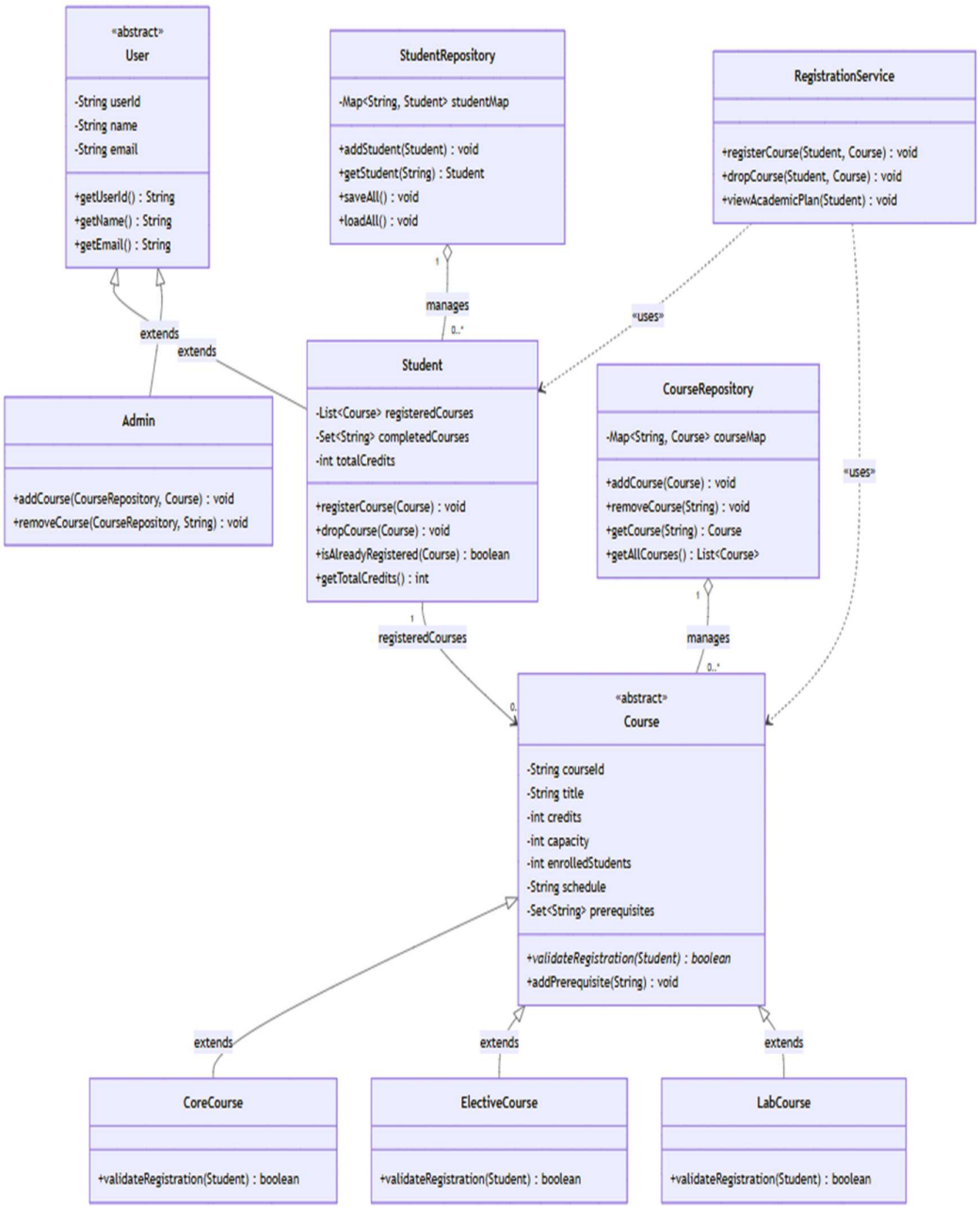
In many academic institutions, course registration systems lack proper validation mechanisms to enforce constraints such as credit limits, prerequisite completion, timetable conflicts, and course capacity. This often leads to inefficient academic planning, scheduling conflicts, and invalid course selections by students.

The aim of this project is to develop a Smart Course Registration and Academic Planner System using object-oriented programming principles. The system ensures that students can only register for courses that satisfy all academic constraints. It dynamically validates credit limits, checks prerequisite completion, prevents timetable clashes, and enforces course capacity restrictions.

The system is designed using a layered architecture that separates data models, business logic, user interface, and data persistence. It demonstrates key OOP concepts such as abstraction, inheritance, polymorphism, exception handling, and collections while simulating a realistic academic registration workflow.

SYSTEM DESIGN

1.UML Class Diagram



Description of Major Classes:

User (Base Class)

- Contains common attributes: userId, name, email
- Acts as parent for Student and Admin

Student

- Maintains registered courses and completed courses
- Tracks total credits
- Supports course registration logic

Admin

- Inherits from User
- Responsible for managing courses
- Can add, remove, and view courses

Course (Abstract Class)

- Common attributes: courseId, title, credits, capacity, schedule
- Declares abstract method validateRegistration()

CoreCourse

- Implements prerequisite validation

ElectiveCourse

- Open to all students (no restrictions)

LabCourse

- Includes labSlots constraint

CourseRepository

- Stores courses using a Map
- Supports add/remove/retrieve operations

StudentRepository

- Handles student persistence

RegistrationService

- Core business logic
- Handles validations: -Credit limit
  -Capacity
  -Time conflict
  -Prerequisites

Package Structure

```
courseplanner/
│
├── model/
│   ├── User.java
│   ├── Student.java
│   ├── Admin.java
│   ├── Course.java
│   ├── CoreCourse.java
│   ├── ElectiveCourse.java
│   ├── LabCourse.java
│
├── repository/
│   ├── CourseRepository.java
│   ├── StudentRepository.java
│
├── service/
│   ├── RegistrationService.java
│
├── exception/
│   ├── CreditLimitExceededException.java
│   ├── PrerequisiteNotMetException.java
│   ├── CourseCapacityFullException.java
│   ├── TimeTableConflictException.java
│   ├── DuplicateCourseException.java
|   ├── CourseNotRegisteredException.java
└── ui/
    ├── MainMenu.java
```

Interaction Flow

1. User logs in using Student ID
2. System loads or creates student data
3. Admin can add/remove courses
4. Student views available courses
5. Student registers for a course
6. System validates constraints:

- Credit limit
- Prerequisites
- Timetable conflict
- Capacity

7. If valid → course registered
8. Student can drop courses
9. Student can view academic plan
10. Data is saved using file handling

OOP CONCEPTS DEMONSTRATED

1. Abstraction

- **Where**: User and Course classes (abstract)
- **Why**: To define a common structure while allowing specific implementations

Code Snippet:

```
public abstract class Course {

  protected String courseId;

  protected String title;


  public abstract boolean validateRegistration(Student student);

}
```

Explanation: The abstract class ensures all course types implement their own validation logic.

2. Inheritance

- **Where**: Student & Admin extend User, Course subclasses extend Course
- **Why**: To reuse common attributes

Code Snippet:

```
public class Student extends User {

  }
```

Explanation: Avoids duplication of user details across classes.

3. Polymorphism

- **Where:** validateRegistration() method in different course types
- **Why:** Different course types behave differently

Code Snippet:

```
@Override

public boolean validateRegistration(Student student) throws PrerequisiteNotMetException {

for (String pre : prerequisites) {

 if (!student.getCompletedCourses().contains(pre)) {

throw new PrerequisiteNotMetException("Missing prerequisite: " + pre);

   }

}

return true;

}
```

Explanation: Same method behaves differently based on object type.

4. Exception Handling

- **Where:** RegistrationService
- **Why:** To enforce academic rules safely

Code Snippet:

```
if(student.getTotalCredits() + course.getCredits() > 20){

  throw new CreditLimitExceededException("Limit exceeded");

}
```

Explanation: Prevents invalid registration and preserves system robustness.

5. Encapsulation

•**Where**: Student class manages its own course list

•**Why**: To protect internal data and ensure control access

Code Snippet:

```
public void registerCourse(Course course) {
 registeredCourses.add(course);
}
```

Explanation: Provides fast lookup and scalability

6. Collections

- **Where:** CourseRepository
- **Why:** Efficient storage and retrieval

Code Snippet:

```
private Map<String, Course> courses = new HashMap<>();
```

Explanation: Provides fast lookup and scalability.

IMPLEMENTATION HIGHLIGHTS

1. Course Registration Logic (Core Validation Engine)

```
public void registerCourse(Student student, Course course)
    throws CreditLimitExceededException,
        CourseCapacityFullException,
        TimeTableConflictException,
        PrerequisiteNotMetException {
  if (student.getTotalCredits() + course.getCredits() > 20) {
    throw new CreditLimitExceededException("Credit limit exceeded");
  }
 if (course.getEnrolledStudents() >= course.getCapacity()) {
    throw new CourseCapacityFullException("Course capacity full");
  }
  for (Course c : student.getRegisteredCourses()) {
```

```
      if (c.getSchedule().equals(course.getSchedule())) {
        throw new TimeTableConflictException("Schedule conflict");
      }
    }
  if (!course.validateRegistration(student)) {
      throw new PrerequisiteNotMetException("Prerequisite not met");
    }
    student.getRegisteredCourses().add(course);
    student.addCredits(course.getCredits());
    course.incrementEnrollment();
}
```

Justification: This is the **heart of the system**. It enforces all academic constraints dynamically, ensuring only valid registrations occur. It also demonstrates exception handling + polymorphism together.

2. Abstract Course Design

```
public abstract class Course {
    protected String courseId;
    protected String title;
    protected int credits;
    protected int capacity;

    public abstract boolean validateRegistration(Student student);
}
```

Justification: This enables flexibility and extensibility. Different course types (Core, Elective, Lab) implement their own validation rules using polymorphism.

3. Repository using Collections

```
public class CourseRepository {
  private Map<String, Course> courses = new HashMap<>();

  public void addCourse(Course course) {
    courses.put(course.getCourseId(), course);
  }

  public Collection<Course> getAllCourses() {
    return courses.values();
  }
}
```

Justification: Demonstrates use of Java Collections (Map) for efficient storage and retrieval. Supports scalability and fast lookups.

TESTING & ERROR HANDLING

Test Cases:

- Valid registration
- Exceeding credit limit
- Registering without prerequisites
- Time clash
- Full capacity

Edge Cases:

- Duplicate course registration
- Empty course list
- Failure Scenarios
- Exception thrown with proper message
- System continues execution

Failure Scenarios:

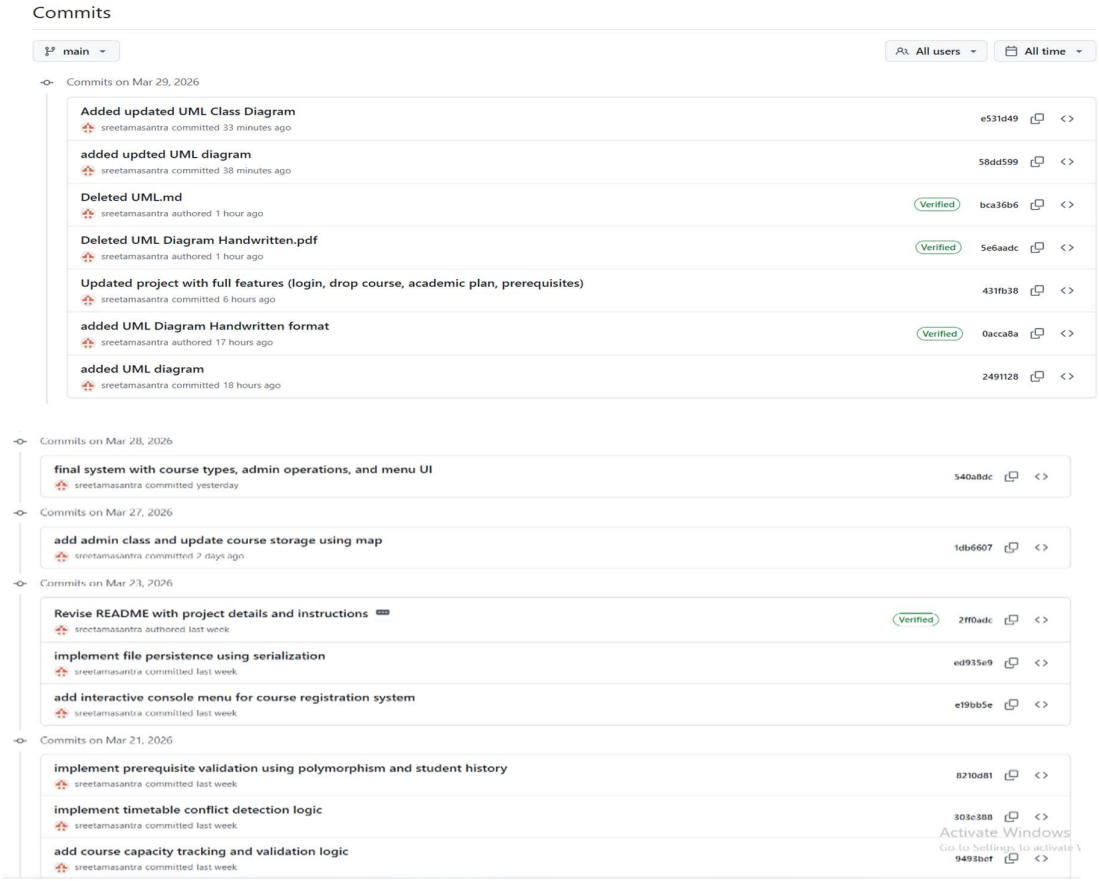
- Exceptions are thrown with clear messages
- Errors are handled in UI layer
- System continues execution without crashing

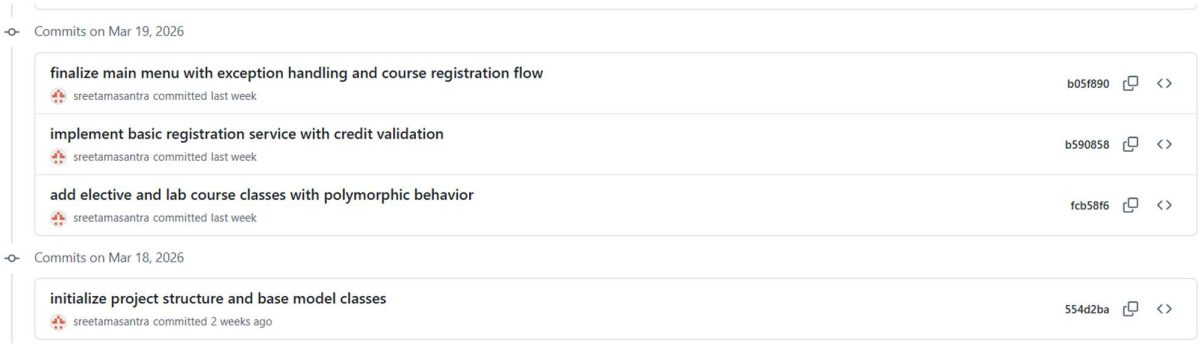
GIT WORKFLOW

- Project initialized using Git
- Multiple commits made during development
- Changes pushed to GitHub repository

Development Progression:

- Initial structure creation
- Model and service implementation
- Exception handling added
- UI and persistence integrated
- Final refactoring and testing





CONCLUSION & FUTURE SCOPE

The Smart Course Registration & Academic Planner System successfully demonstrates the use of object-oriented principles in building a modular and scalable application. It ensures efficient academic planning by enforcing rules such as credit limits, prerequisites, and timetable constraints.

Future Enhancements:

- GUI using JavaFX or Swing
- Database integration
- Web-based system using Spring Boot
- Advanced timetable management

Appendix

Copy of project proposal: https://github.com/IEM-Gurukul/term-ii-project-submission-sreetamasantra/blob/f5ef1bda42fd5b023bfe852815c01462448089c9/docs/java%20project%20proposal.pdf

Copy of Approval email: https://github.com/IEM-Gurukul/term-ii-project-submission-sreetamasantra/blob/b1b25801bbccacc24036a485969b6d6072b9afcd/docs/copy%20of%20approval%20email.pdf